MBARARA UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF COMPUTING AND INFORMATICS

DEPARTMENT OF SOFTWARE ENGINEERING



SOFTWARE REQUIREMENTS SPECIFICATION

Employee Recruitment System (ERS)

A Long-Duration Business Transaction Processing System, from Vacancy Creation to Employee Onboarding

COURSE:	CLIENT SERVER PROGRAMMING FOR APPLICATIONS
COURSE CODE:	SWE4107
SEMESTER:	1
ACADEMIC YEAR:	2026 / 2027
YEAR OF STUDY:	FOUR

GROUP MEMBERS

No.	Full Name	Registration Number
1	AYEBARE PATIENCE PRAISE	2023/BSE/037/PS
2	AMWINE LIAM ABAASA	2023/BSE/018/PS
3	OGWAL BILL EDWIN	2023/BSE/126/PS
4	AGUMYA NYSON	2023/BSE/005/PS
5	KAMUKAMA SIMON PETER	2023/BSE/059/PS

No.	Full Name	Registration Number
6	MUGUMYA EDWIN	2023/BSE/087/PS
7	NUWE BRYANT NYERO	2023/BSE/122/PS
8	OKIROR MARK	2023/BSE/130/PS
9	NATUKUNDA JOVITA	2023/BSE/109/PS

Submitted in partial fulfilment of the semester-long group project for SWE4107, a working document to guide the group through requirements analysis, design, and implementation of the Employee Recruitment System.

Abstract

Hiring an employee is not one action; it is a chain of separate activities, advertising a vacancy, receiving applications, screening, interviewing, evaluating, approving, offering, and onboarding, carried out by different people at different times, sometimes weeks apart. A conventional CRUD application, built around instant create/read/update/delete operations, has no natural way to represent a process that stays open for that long, involves that many actors, and must remember exactly where it left off every time someone returns to it.

This document is the Software Requirements Specification (SRS) for the Employee Recruitment System (ERS), the semester-long working document for the SWE4107 Client-Server Programming for Applications group project. ERS is modelled explicitly as a long-duration business transaction: one recruitment record moves through a defined sequence of states, SUBMITTED, UNDER_SCREENING, SHORTLISTED, INTERVIEW_SCHEDULED, UNDER_EVALUATION, SELECTED, PENDING_APPROVAL, OFFER_SENT, OFFER_ACCEPTED, ONBOARDING, COMPLETED, with REJECTED and DECLINED branches at several points, and the system's central responsibility is to persist that state reliably across sessions, days, and weeks, and to make it visible to whichever authorised actor needs to act on it next.

The system is client-server: separate client experiences for the Candidate, the HR Officer, the Interview Panel, the Department Manager, and the System Administrator all talk to one application server over HTTPS/JSON, which in turn persists every stage of every recruitment transaction to a relational database. This document covers the requirements analysis behind that design, the business rules that keep a multi-week hiring process auditable and recoverable, the functional and non-functional requirements organised around the recruitment lifecycle, the entity-relationship design of the underlying schema, and the interface, security, and testing requirements the group will build against for the remainder of the semester.

Keywords: Long-Duration Transactions, Business Process State Management, Recruitment and Hiring, Client-Server Architecture, FastAPI, PostgreSQL, React.

Acronyms

Acronym	Meaning
ERS	Employee Recruitment System
SRS	Software Requirements Specification
HR	Human Resources
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
JWT	JSON Web Token
ERD	Entity Relationship Diagram
PK / FK	Primary Key / Foreign Key
TLS	Transport Layer Security
UAT	User Acceptance Testing
RDBMS	Relational Database Management System

Table of Contents

Abstract.....2

Acronyms.....3

Table of Contents.....4

1. Introduction and Background.....6

1.1 Purpose of the System.....6

1.2 Scope of the System.....6

1.3 Objectives.....6

Main Objective.....6

Specific Objectives.....6

1.4 Problem Statement.....7

2. Overall Description.....8

2.1 Existing System.....8

2.2 Proposed System.....8

2.3 Entity and Domain Description.....8

2.4 System Users.....8

2.5 Operating Environment.....9

2.6 Design Constraints.....9

2.7 Business Rules.....9

3. System Requirements.....11

3.1 Functional Requirements.....11

3.1.1 Authentication & Vacancy Management.....11

3.1.2 Candidate Application.....11

3.1.3 Screening, Shortlisting & Interviews.....11

3.1.4 Selection, Approval & Offer.....12

3.1.5 Onboarding & Administration.....12

3.2 Non-Functional Requirements.....12

3.2.1 Performance, Reliability & Security.....12

3.2.2 Auditability, Usability & Maintainability.....13

3.3 Recruitment Transaction State Model.....13

4. System Design.....14

4.1 Architectural Design.....14

4.2 Database Design..... 14

5. Interface, Security, and Testing Requirements..... 17

5.1 Interface Requirements..... 17

5.1.1 User Interface..... 17

5.1.2 Hardware Interface..... 17

5.1.3 Communication Interface..... 17

5.2 System Security Requirements..... 17

5.3 Testing Requirements..... 17

6. Appendices..... 19

Appendix A: Sample Forms..... 19

A.1 Vacancy Creation Form..... 19

A.2 Interview Evaluation Form..... 19

Appendix B: Sample Questionnaire (Requirements Gathering)..... 19

Appendix C: Technology Stack..... 19

References..... 21

1. Introduction and Background

1.1 Purpose of the System

The Employee Recruitment System (ERS) exists to take the hiring process out of scattered emails, spreadsheets, and physical files and turn it into one continuous, trackable record per candidate. From the moment a vacancy is advertised to the moment a hired candidate completes onboarding, ERS remembers exactly where that recruitment stands, who acted on it last, and who needs to act on it next, even when weeks pass between those actions.

This document is also the working document referred to in the SWE4107 assignment brief: it is what the group uses to agree, in writing, on the requirements, the architecture, the database design, and the technology choices before implementation begins, and it is what the group returns to throughout the semester as the system is built.

1.2 Scope of the System

ERS covers the full recruitment lifecycle: vacancy creation and publication, candidate application, screening and shortlisting, interview scheduling and panel evaluation, selection, management approval, offer generation and response, and onboarding document collection through to recruitment completion. It provides distinct client experiences for the Candidate, the HR Officer, the Interview Panel, the Department Manager, and the System Administrator, all backed by one persistent server-side record per recruitment transaction.

Out of scope: payroll processing and ongoing employee management once onboarding is marked COMPLETED, learning-management or training-content delivery, and the internal workings of any third-party background-verification service beyond recording its outcome against the application record.

1.3 Objectives

Main Objective

To design and implement, over the course of the semester, a client-server Employee Recruitment System that models recruitment as a long-duration business transaction, a single record that persists through multiple stages, multiple actors, and multiple waiting periods, rather than as a simple create/read/update/delete application.

Specific Objectives

1. To document the requirements, business rules, and state model of the recruitment lifecycle before any implementation begins, per this SRS.

2. To design a relational schema and application layer that persist a recruitment transaction's current state reliably across sessions, so that no candidate's progress is lost between the actions of different actors on different days.
3. To build distinct client interfaces, Candidate, HR, Interview Panel, Manager, and Administrator, each showing only the recruitment transactions and actions relevant to that role, with wireframes/mockups to be added as the group's design work progresses.
4. To test the system's state-transition logic, its handling of rejection and decline branches, and its persistence across simulated multi-day gaps, targeting at least 90% of defined test cases passing before submission.

1.4 Problem Statement

A conventional web application is built around instant operations: a form is submitted, a row is written, the interaction is over. Recruitment does not fit that shape. Hiring one candidate can take days or weeks, passes through the hands of several different people (an HR officer, an interview panel, a department manager), and regularly has to wait, for an interview slot, for a manager's decision, for a candidate's response to an offer, without losing track of anything in between. An organisation that runs this process on email threads and spreadsheets ends up with the same recurring problems: no single place that shows where a given candidate's application actually stands, evaluations and approvals recorded inconsistently or not at all, offers that go unanswered without anyone noticing, and no retrievable history explaining why a candidate was rejected at a given stage.

ERS is scoped specifically to close that gap: it treats recruitment as what it structurally is, a long-duration, multi-actor, multi-stage business transaction, and gives it the persistent state management and audit trail that pattern requires, instead of forcing it into a model built for operations that finish in one sitting.

2. Overall Description

2.1 Existing System

There is presently no dedicated system for this process. Vacancies are advertised informally, applications arrive by email or on paper, and screening decisions live in whichever HR officer's inbox or spreadsheet they were made in. Interview scheduling is coordinated by phone and calendar invite, evaluations are recorded inconsistently (sometimes on paper, sometimes not at all), and management approval is often a verbal or emailed yes/no with no permanent record attached to the candidate's file. Once an offer is sent, tracking whether and when the candidate responded depends on someone remembering to follow up. The result is candidates who fall through the cracks, decisions with no retrievable justification, and no single view of where any given recruitment actually stands.

2.2 Proposed System

ERS replaces that informal chain with one persistent transaction per candidate application, visible in real time to every authorised actor who needs to see it. An HR Officer creates and publishes a vacancy once; from that point, each application that comes in is tracked through screening, shortlisting, interview, evaluation, selection, approval, offer, and onboarding as a single continuous record. Every stage transition is stored with who performed it and when, every evaluation and approval decision is attached permanently to the application, and every actor, candidate included, can see exactly where a given application stands without needing to ask anyone.

2.3 Entity and Domain Description

Application is the long-duration transaction itself: one candidate's pursuit of one vacancy, from SUBMITTED through to COMPLETED, REJECTED, or DECLINED. It carries a single, authoritative CurrentState value at any point in time, and its full state history is retrievable even after it closes.

Job (Vacancy) is the position being recruited for, its title, department, required skills, and closing date. One Job can receive many Applications.

Interview is a scheduled event tied to one Application; it exists independently of the Evaluations submitted against it, since a panel of several people may each submit a separate evaluation for the same interview.

Evaluation is one panelist's scored assessment of a candidate following an Interview. Several Evaluations can exist for a single Interview, one per panel member.

Approval is the Department Manager's recorded decision on a Selected candidate. It is distinct from an Evaluation: an Evaluation is a panel member's technical judgement; an Approval is management's authority to proceed to an offer.

Offer captures the terms extended to a candidate (salary, start date, contract type) and the candidate's response. It is the point at which the transaction depends on an external party, the candidate, to move forward.

2.4 System Users

User Type	Description	Primary Interactions
Candidate	Applies for advertised vacancies and responds to offers	Registers, applies, uploads CV, tracks application status, views interview details, accepts/declines offer
HR Officer	Owns the recruitment process day to day	Creates and publishes vacancies, screens and shortlists applications, schedules interviews, generates offers, manages onboarding documents
Interview Panel Member	Evaluates shortlisted candidates	Views assigned interviews and candidate information, submits scored evaluations
Department Manager	Holds hiring approval authority	Reviews selected candidates and recommendations, approves or rejects hiring
System Administrator	ICT personnel responsible for platform upkeep	Manages users, roles, and departments; monitors transactions; generates reports

2.5 Operating Environment

ERS is a browser-based client-server application built on a three-tier architecture. The presentation layer is a single-page application built with React and styled with Tailwind CSS, presenting distinct views for each of the five actor roles, served to any modern browser without requiring installed client software. The application layer is built with Python and the FastAPI framework, exposing a REST/JSON API that enforces the state-transition rules governing which role may move an application from one stage to the next. The data layer is a PostgreSQL relational database holding the entities defined in Section 4.2, chosen specifically for its ability to preserve, query, and report on a long-running record's full history reliably. The system is intended to run on standard Linux server infrastructure for the duration of the semester project.

2.6 Design Constraints

- Recruitment is modelled as one long-duration business transaction per application, not as a series of unrelated CRUD records; every screen and endpoint must read and write against that transaction's persistent state rather than reconstructing status from scattered records.
- A recruitment transaction must be resumable by any authorised actor at any later time, including after gaps of days or weeks, without loss of prior state or history.
- Every stage transition must be attributable to a specific authenticated user and role; no state change may be recorded without an identifiable responsible actor.
- Development is constrained to one academic semester, which limits the system to a functional client-server prototype covering the core recruitment lifecycle rather than a fully hardened production deployment.
- The group's chosen technology stack, React/Tailwind CSS on the frontend, Python/FastAPI on the backend, and PostgreSQL for persistence, is fixed for the duration of the project (see Section 4.1 and Appendix D).

2.7 Business Rules

These rules govern the recruitment lifecycle independently of any single screen or module; Chapter 3 refers back to them by ID wherever a requirement enforces one directly.

ID	Rule
BR-01	Only authenticated users may access any part of the system, and access is restricted according to the user's assigned role.
BR-02	A recruitment transaction's current state shall be persisted to the database at every stage change, so that no progress is lost between sessions or between the actions of different actors.
BR-03	A candidate may only be advanced to the next stage by a user holding the role authorised for that stage (e.g., only an Interview Panel member may submit an evaluation; only a Department Manager may approve or reject a hiring recommendation).
BR-04	Every state transition shall be recorded with a timestamp and the identity of the responsible actor; history is preserved, never overwritten.
BR-05	A rejection or decline at any stage shall close that path for the candidate but shall not delete the application record; the full history up to that point remains retrievable.
BR-06	An offer sent to a candidate shall remain in OFFER_SENT until the candidate responds or a defined response deadline lapses; the system shall not lose or silently expire an unanswered offer.
BR-07	Every approval or rejection decision made by a Department Manager shall be logged with the approver's identity, the decision, and a timestamp.

ID	Rule
BR-08	A recruitment transaction cannot be marked COMPLETED until all required onboarding documents are verified and the relevant IT, Finance, and Department confirmations are recorded.

3. System Requirements

Each requirement below maps to a stage of the recruitment lifecycle and, where relevant, to a Business Rule from Section 2.7, so there is a clean line of traceability from actor to lifecycle stage to requirement to acceptance test.

3.1 Functional Requirements

3.1.1 Authentication & Vacancy Management

ID	Requirement	Acceptance Criteria
FR-01	**Role-Based Login** Authenticate all users by username and password; restrict screens and actions by role (Candidate, HR Officer, Panel, Manager, Administrator), per BR-01.	• No screen beyond login is reachable without valid credentials. • A Candidate cannot reach HR, Panel, Manager, or Admin screens.
FR-02	**Create & Publish Vacancy** Allow an HR Officer to create a vacancy (title, department, positions, skills, closing date) and publish it for candidates to view and apply to.	• A vacancy cannot publish without title, department, and closing date. • A published vacancy is immediately visible to candidates.

3.1.2 Candidate Application

ID	Requirement	Acceptance Criteria
FR-03	**Submit Application** Allow a registered Candidate to apply to a published vacancy with personal details, CV, qualifications, experience, and references; persist as SUBMITTED, per BR-02.	• An application cannot submit without a CV attachment. • The application is retrievable after logout/login later.
FR-04	**Track Application Status** Allow a Candidate to view their application's current state and stage history at any time.	• Status shown always matches the authoritative CurrentState in the database. • A candidate can check status without contacting HR.

3.1.3 Screening, Shortlisting & Interviews

ID	Requirement	Acceptance Criteria
FR-05	**Screen & Shortlist / Reject**	• A rejected application is retained with full history, per BR-05.

ID	Requirement	Acceptance Criteria
	Allow an HR Officer to move SUBMITTED applications to SHORTLISTED or REJECTED, per BR-03.	• Only an HR Officer role can perform this transition.
FR-06	**Schedule Interview** Allow an HR Officer to schedule an interview (date, time, location, panel members) for a SHORTLISTED application, moving it to INTERVIEW_SCHEDULED.	• Assigned panel members see the interview immediately. • An interview cannot be assigned to an unregistered panel member.
FR-07	**Submit Panel Evaluation** Allow each assigned Panel member to submit a scored evaluation (technical, communication, problem solving, teamwork, comments) after an interview, per BR-03.	• Each panelist's evaluation is recorded and attributed separately. • The application advances to UNDER_EVALUATION only once the interview is marked completed.

3.1.4 Selection, Approval & Offer

ID	Requirement	Acceptance Criteria
FR-08	**Select Candidate** Allow HR/Panel to move an evaluated application to SELECTED or REJECTED based on recorded scores.	• A SELECTED application automatically routes to the assigned Manager. • Evaluation scores remain attached to the record.
FR-09	**Manager Approval** Allow a Department Manager to review a SELECTED candidate and record APPROVED or REJECTED, per BR-07.	• A decision cannot record without an identifiable Manager and timestamp. • An APPROVED application becomes eligible for an offer.
FR-10	**Generate Offer & Capture Response** Allow HR to generate an offer (salary, start date, contract) for an APPROVED candidate and record ACCEPTED/DECLINED, per BR-06.	• The offer stays visible as OFFER_SENT until a response is recorded. • ACCEPTED automatically moves the application into ONBOARDING.

3.1.5 Onboarding & Administration

ID	Requirement	Acceptance Criteria
FR-11	**Onboarding Document Collection**	• COMPLETED cannot be reached with any confirmation outstanding.

ID	Requirement	Acceptance Criteria
	Allow HR, IT, Finance, and the Department to record their onboarding confirmations; mark COMPLETED only once all are confirmed, per BR-08.	• Each department's confirmation is attributed to the user who recorded it.
FR-12	**Manage Users, Roles & Reports** Allow a System Administrator to manage user accounts and roles, monitor in-progress transactions, and generate recruitment reports.	• A deactivated account cannot log in. • Reports filter by date range, department, and vacancy.

3.2 Non-Functional Requirements

3.2.1 Performance, Reliability & Security

ID	Requirement	Acceptance Criteria
NFR-01	**Response Time** Application-status and dashboard queries return within 3 seconds under normal load.	• 95% of status queries complete within 3 seconds.
NFR-02	**State Persistence Across Sessions** Every transaction's state is retrievable unchanged after any gap in activity, including restarts, per BR-02.	• Simulated multi-week gaps show no loss of state or history. • No state depends on data held only in a client session.
NFR-03	**Password Encryption** User passwords are stored using a one-way hash (e.g., bcrypt); never stored or transmitted in plain text.	• The USERS table shows only hashed values. • Password reset generates a new, non-recoverable hash.
NFR-04	**Role-Based Authorisation** Access control is enforced server-side, independent of what the client interface displays, per BR-03.	• An unauthorised request to the FastAPI backend is rejected server-side. • Every authorisation failure is logged.
NFR-05	**Data Transmission Security** All client-server traffic is encrypted using HTTPS/TLS.	• Plain-HTTP access is redirected to HTTPS.

3.2.2 Auditability, Usability & Maintainability

ID	Requirement	Acceptance Criteria
NFR-06	**Auditability** An immutable record is kept of every stage transition, evaluation, and approval decision, per BR-04 and BR-07.	• Audit entries cannot be edited or deleted by any role. • A closed application's full history remains retrievable.
NFR-07	**Interface Clarity** Each client shows only the information and actions relevant to its role, interpretable without extra training.	• A first-time HR Officer locates an application's stage within 3 minutes, unaided.
NFR-08	**Modular Architecture** The recruitment workflow/state-transition logic is a distinct FastAPI module, separate from user-management and reporting code.	• A new stage or rule can be added without touching authentication or reporting.

3.3 Recruitment Transaction State Model

The diagram in the original example SRS used a use case and swimlane image; this working document instead records the same information as a compact text-based state flow, since it maps directly onto the CurrentState column of the APPLICATION table.

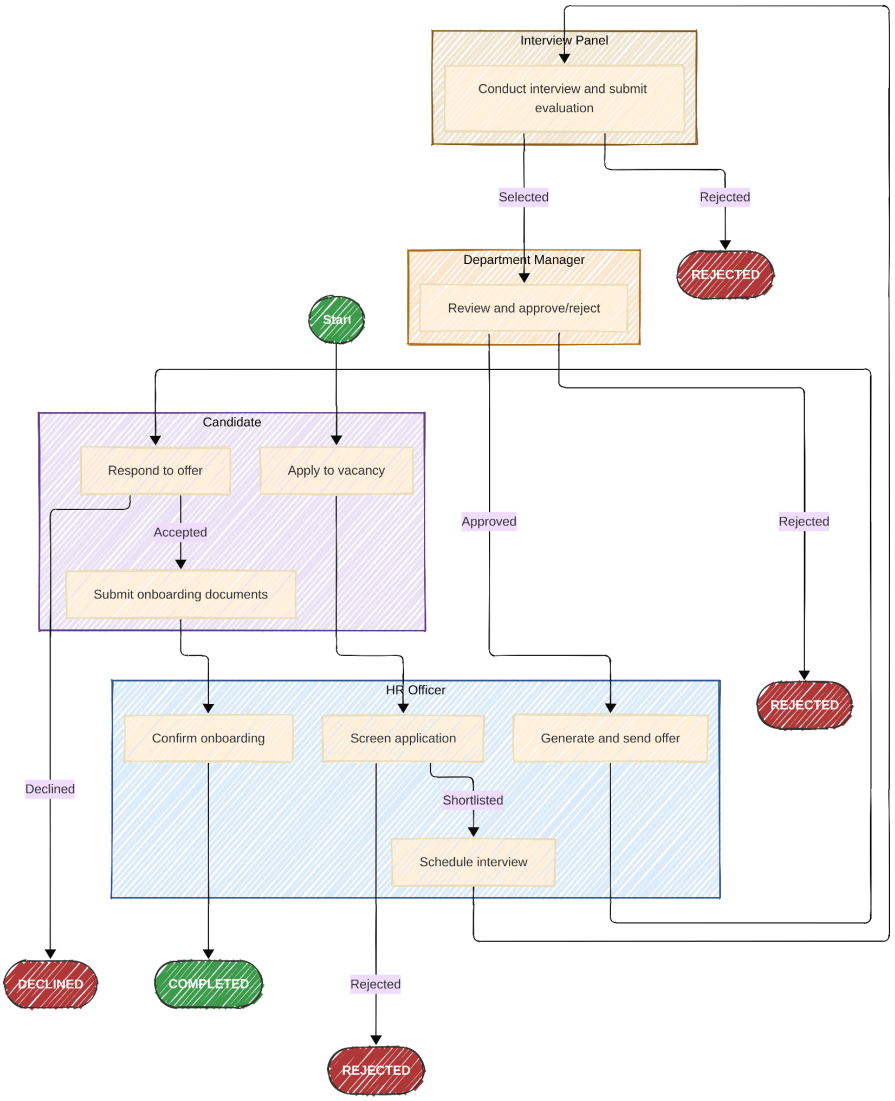


Figure 3.1: ERS Business Process: Cross-Functional Swimlane Diagram

Every branch ending in REJECTED or DECLINED is a closed transaction, not a deleted one, its full history up to that point remains queryable, per BR-05.

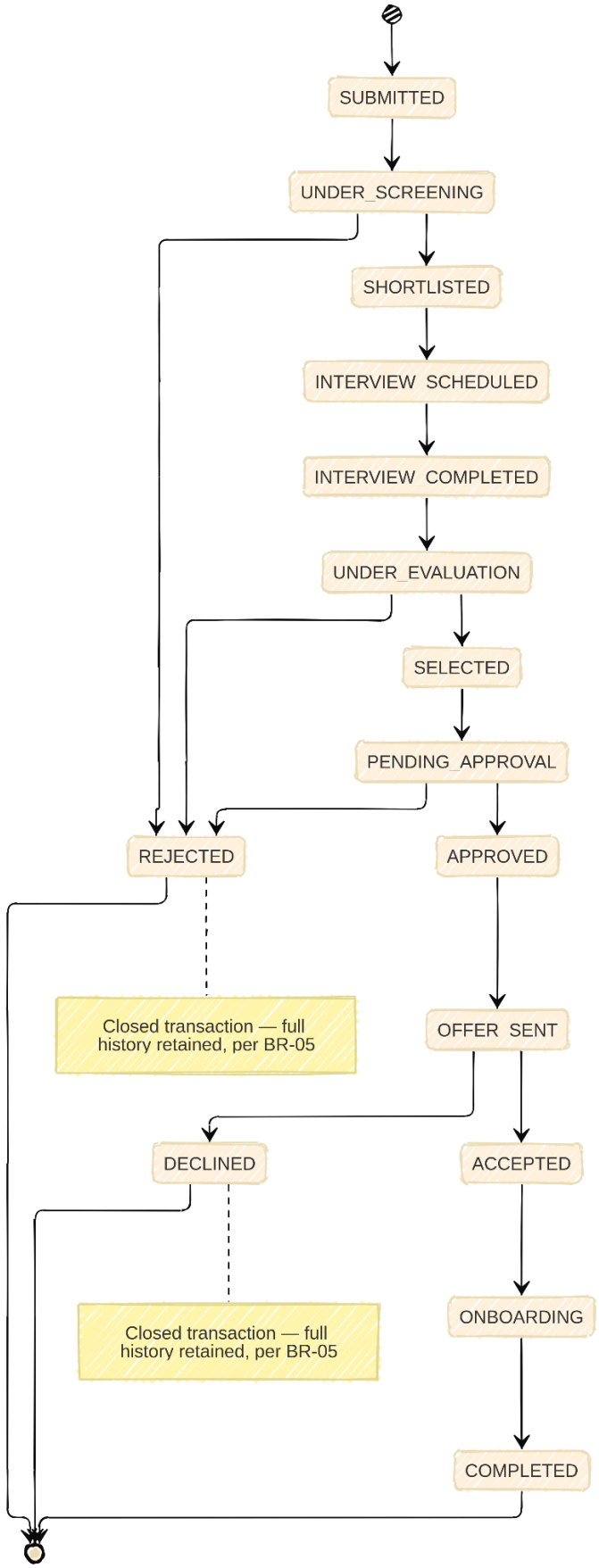


Figure 3.2: ERS Recruitment Transaction State Model

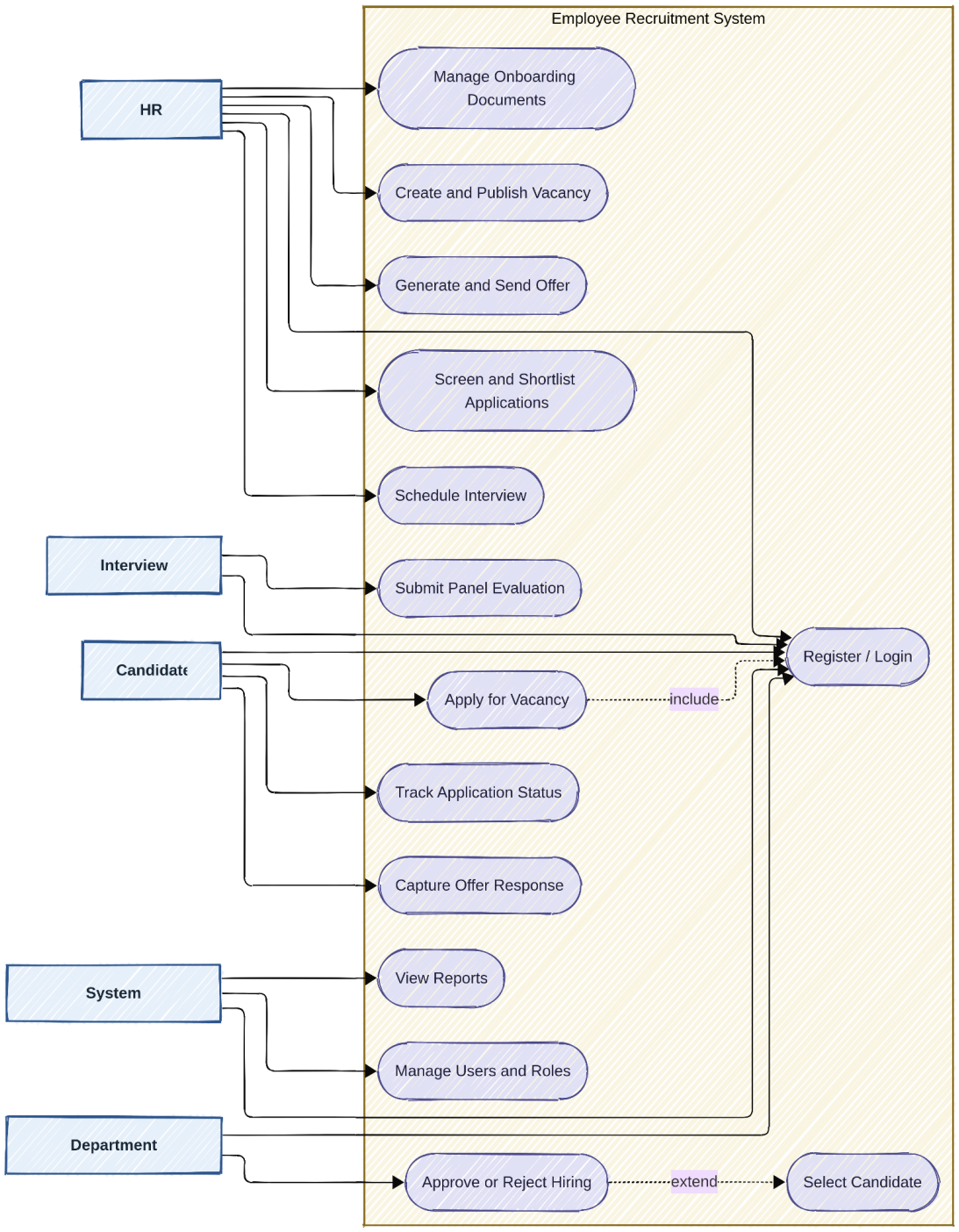


Figure 3.3: ERS Use Case Diagram

4. System Design

4.1 Architectural Design

ERS follows a three-tier client-server architecture. The group has fixed the following technology stack for the duration of the project, per the assignment's requirement that each group specify its programming languages and database management system:

- Presentation Layer: JavaScript, HTML, and CSS, built as a single-page application with React and styled with Tailwind CSS. Five distinct view sets are served from the same codebase, Candidate, HR Officer, Interview Panel, Department Manager, and System Administrator, gated by role after login.
- Application / Logic Layer: Python, using the FastAPI framework for the REST/JSON API. This layer owns the recruitment workflow engine, which validates that a requested stage transition is both legal for the current state and permitted for the requesting user's role, per BR-03.
- Data Layer: PostgreSQL, holding the ten entities defined in Section 4.2. The APPLICATION table's CurrentState column is the single source of truth for where any recruitment transaction stands; INTERVIEW, EVALUATION, APPROVAL, and OFFER each attach detail to that one authoritative record.

Keeping the workflow engine as one distinct module (per NFR-08) is what lets the group add a new stage or a new role's permissions later in the semester without touching authentication or reporting code.

4.2 Database Design

The ten entities below capture the recruitment lifecycle described in Section 2.3. APPLICATION is the central table: JOB and USERS (as Candidate) feed into it, and INTERVIEW, EVALUATION, APPROVAL, and OFFER each attach to it as the transaction progresses, each carrying its own foreign key back to APPLICATION rather than to one another. ROLES and USERS support authentication and access control; NOTIFICATION and AUDIT_LOG support visibility and traceability.

ROLES		
Field	**Type**	**Key**
RoleID	int	**PK**
RoleName	varchar	
Description	text	
USERS		
Field	**Type**	**Key**

UserID	int	**PK**
Username	varchar	
PasswordHash	varchar	
Email	varchar	
RoleID	int	**FK → ROLES**
AccountStatus	varchar	
CreatedAt	datetime	
JOB		
Field	**Type**	**Key**
JobID	int	**PK**
Title	varchar	
Department	varchar	
Description	text	
NumPositions	int	
RequiredSkills	text	
ClosingDate	date	
Status	varchar	
CreatedBy	int	**FK → USERS**
APPLICATION		
Field	**Type**	**Key**
ApplicationID	int	**PK**
CandidateID	int	**FK → USERS**
JobID	int	**FK → JOB**
CurrentState	varchar	
AppliedAt	datetime	
UpdatedAt	datetime	
INTERVIEW		

Field	Type	Key
InterviewID	int	**PK**
ApplicationID	int	**FK → APPLICATION**
ScheduledDate	datetime	
Location	varchar	
Status	varchar	

EVALUATION

Field	Type	Key
EvaluationID	int	**PK**
InterviewID	int	**FK → INTERVIEW**
PanelistID	int	**FK → USERS**
TechnicalScore	int	
CommunicationScore	int	
ProblemSolvingScore	int	
TeamworkScore	int	
Comments	text	

APPROVAL

Field	Type	Key
ApprovalID	int	**PK**
ApplicationID	int	**FK → APPLICATION**
ApproverID	int	**FK → USERS**
Decision	varchar	
Timestamp	datetime	

OFFER

Field	Type	Key
OfferID	int	**PK**
ApplicationID	int	**FK → APPLICATION**
Salary	decimal	

StartDate	date	
Contract	varchar	
Status	varchar	
ResponseDate	date	
NOTIFICATION		
Field	**Type**	**Key**
NotificationID	int	**PK**
RecipientID	int	**FK → USERS**
Message	text	
Type	varchar	
SentDate	datetime	
Status	varchar	
AUDIT_LOG		
Field	**Type**	**Key**
LogID	int	**PK**
UserID	int	**FK → USERS**
ActionPerformed	varchar	
Timestamp	datetime	
AffectedRecord	varchar	

5. Interface, Security, and Testing Requirements

5.1 Interface Requirements

5.1.1 User Interface

The React/Tailwind interface presents a role-specific view on login: a Candidate sees their applications and current status; an HR Officer sees vacancies and applications awaiting action; an Interview Panel member sees assigned interviews and pending evaluations; a Department Manager sees candidates awaiting approval; a System Administrator sees user management, transaction monitoring, and reports. Every status screen shows not just the current stage but the history that led to it, so no actor has to ask another actor what already happened.

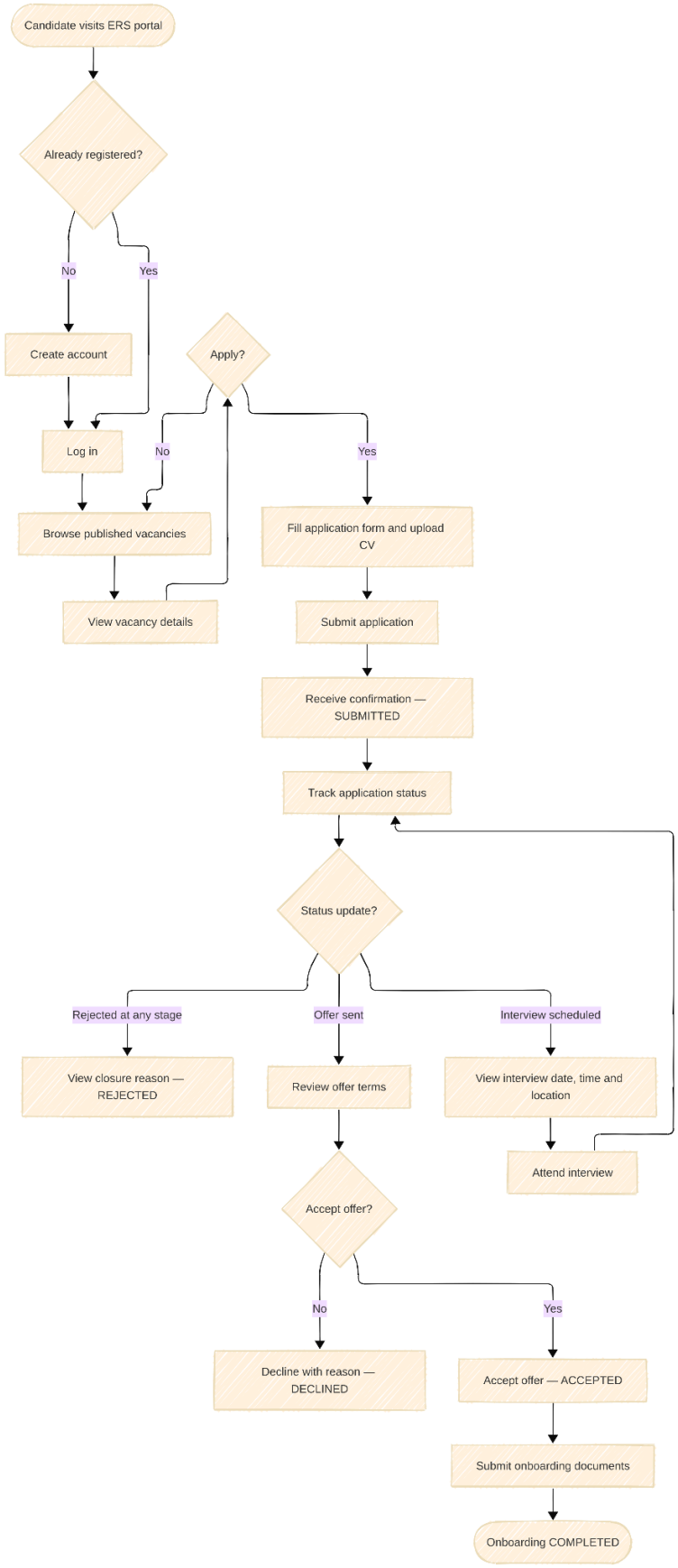


Figure 5.1: ERS Candidate User Flow

5.1.2 Hardware Interface

No specialised hardware is required. The system runs on any device with a modern web browser and standard internet or intranet connectivity.

5.1.3 Communication Interface

The React client communicates with the FastAPI backend over HTTPS using JSON request and response bodies. No third-party webhook integrations are required for the core recruitment lifecycle; where a background-verification or reference-check service is added later in the semester, it would follow the same authenticated REST pattern as the rest of the API.

5.2 System Security Requirements

Authentication: Covered by FR-01: username/password login with role-based redirection.

Authorisation: Covered by NFR-04: role-based access enforced server-side in FastAPI, not just hidden in the React interface.

Password Encryption: Covered by NFR-03: passwords are hashed with a one-way algorithm (bcrypt or equivalent) and never stored or logged in plain text.

Data in Transit: Covered by NFR-05: all client-server traffic is encrypted over HTTPS/TLS.

Auditability: Covered by NFR-06: every stage transition, evaluation, and approval decision is written to an immutable audit trail.

5.3 Testing Requirements

Testing Type	Focus	Example Check
Unit Testing	Individual functions in isolation	The state-transition function correctly rejects an illegal move (e.g., SUBMITTED → APPROVED)
Integration Testing	Modules working together	Scheduling an interview correctly notifies the assigned panel member and updates the application state
State-Transition Testing	Lifecycle correctness under simulated gaps	An application left at UNDER_EVALUATION for a simulated two-week gap resumes correctly with no data loss
Security Testing	Access control	A Candidate session cannot reach a Manager-only approval endpoint by direct request manipulation

Testing Type	Focus	Example Check
System Testing	End-to-end behaviour	The full happy-path flow (application through onboarding completion) completes correctly across all five client roles
User Acceptance Testing	Real users, real judgement	An HR Officer completes an actual recruitment walkthrough and confirms the status view matches their understanding of where the candidate stands

6. Appendices

Appendix A: Sample Forms

A.1 Vacancy Creation Form

Field	Detail
Vacancy ID	Auto-generated
Job Title	
Department	
Number of Positions	
Required Skills	
Closing Date	
Created By	Auto-filled (logged-in HR Officer)

A.2 Interview Evaluation Form

Field	Detail
Evaluation ID	Auto-generated
Interview ID	Auto-linked
Panelist	Auto-filled (logged-in Panel member)
Technical Knowledge Score	0–100
Communication Score	0–100
Problem Solving Score	0–100
Teamwork Score	0–100
Comments	Free text

Appendix B: Sample Questionnaire (Requirements Gathering)

1. How is a candidate's progress currently tracked from application to hiring?
2. On average, how long does the process take from application to a signed offer?
3. How are interview evaluations currently recorded, and by whom?

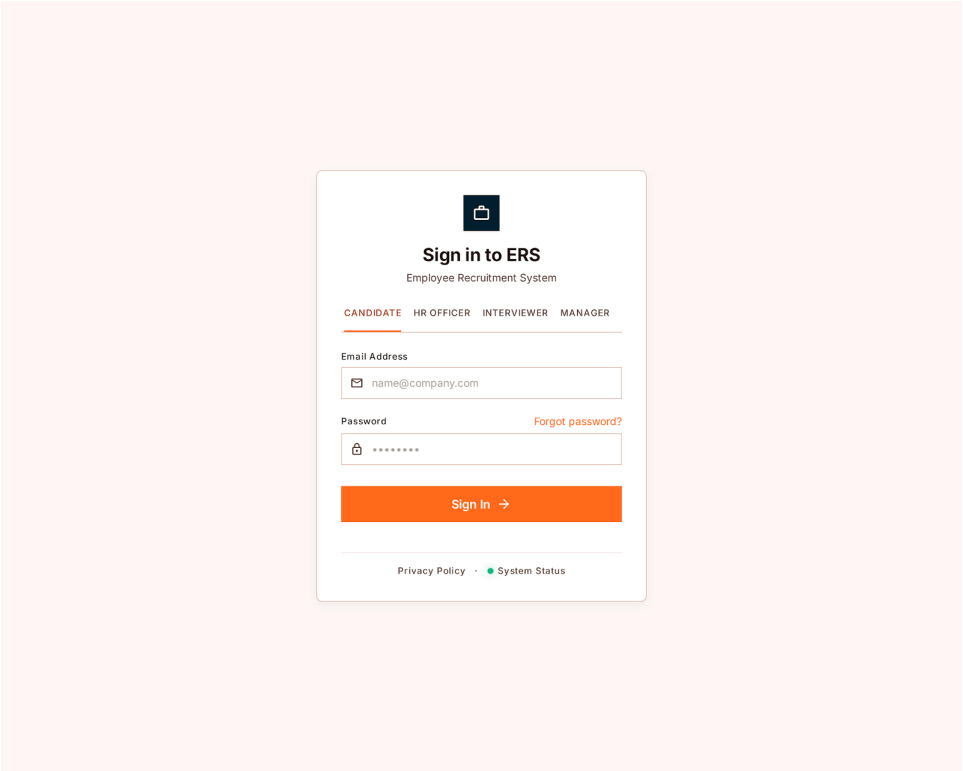
4. Is there a written record of why a given candidate was rejected or approved at each stage?
5. Would a system that shows every actor exactly where a candidate stands change how HR, panel members, and managers coordinate on a hire?

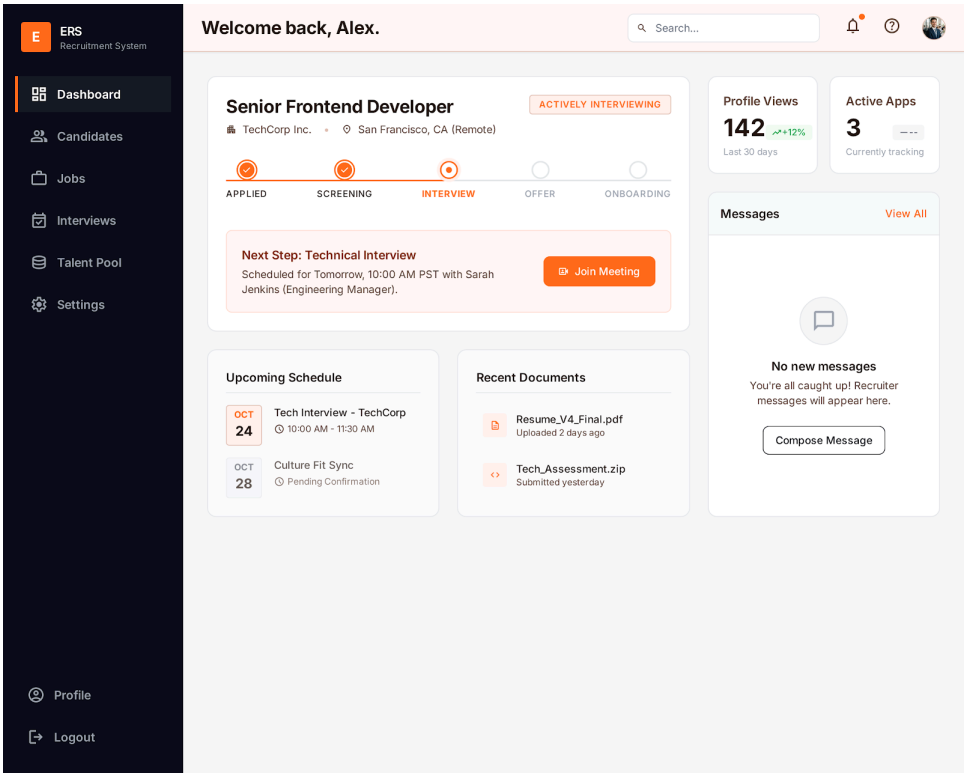
Appendix C: Technology Stack

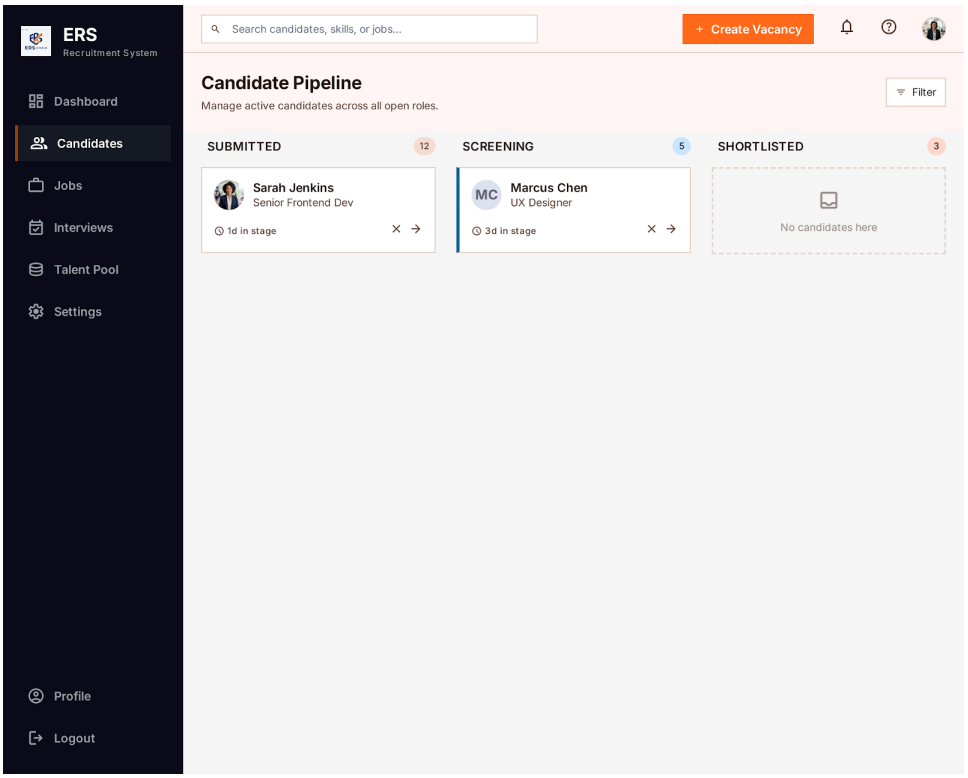
Per the assignment's requirement that each group specify its programming languages and database management system, the group has fixed the following stack:

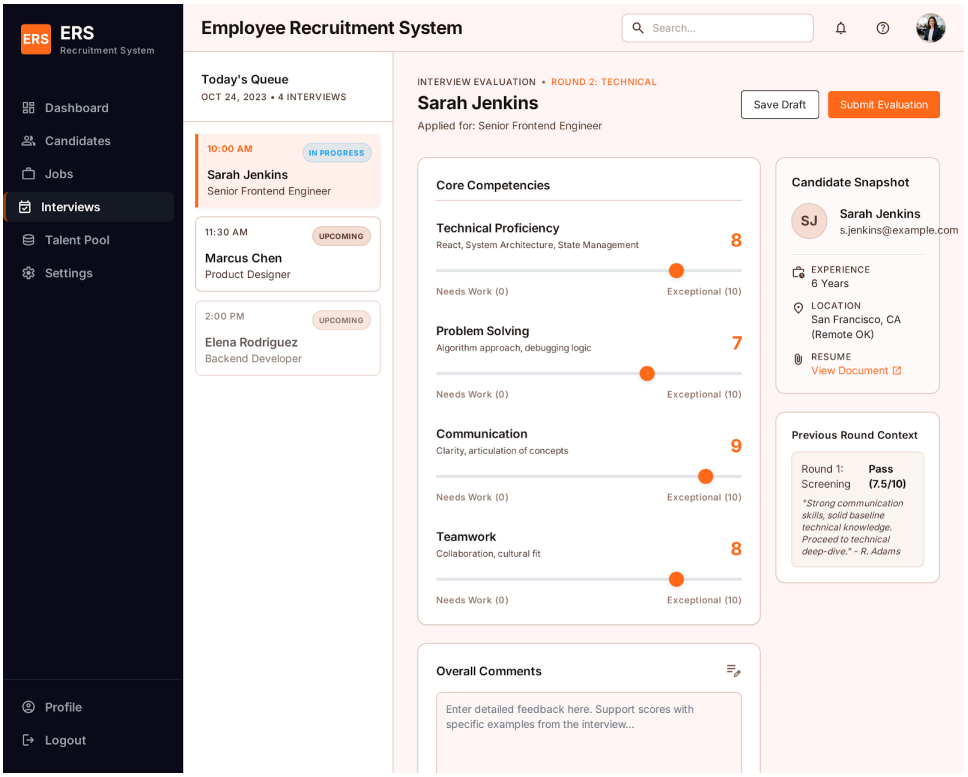
Layer	Technology
Presentation (Frontend)	JavaScript, HTML, CSS, React with Tailwind CSS
Application / Logic (Backend)	Python, FastAPI, hosting the recruitment workflow engine
Data Persistence	PostgreSQL
Authentication	JSON Web Tokens (JWT) issued by the FastAPI backend, validated on every request

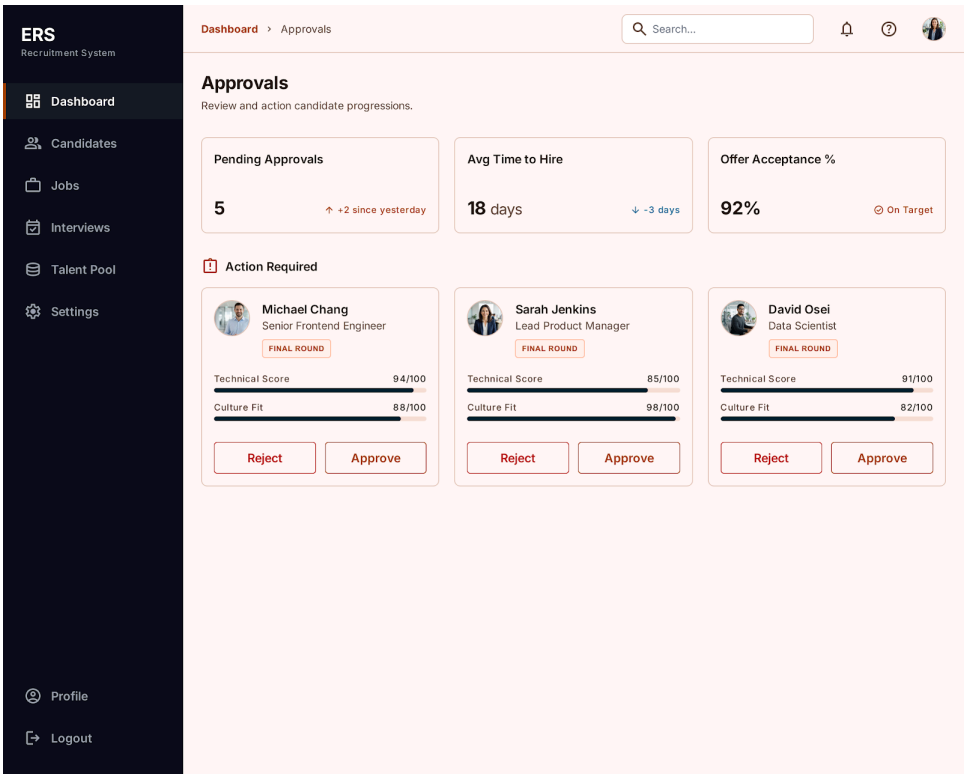
Appendix D: Mockups

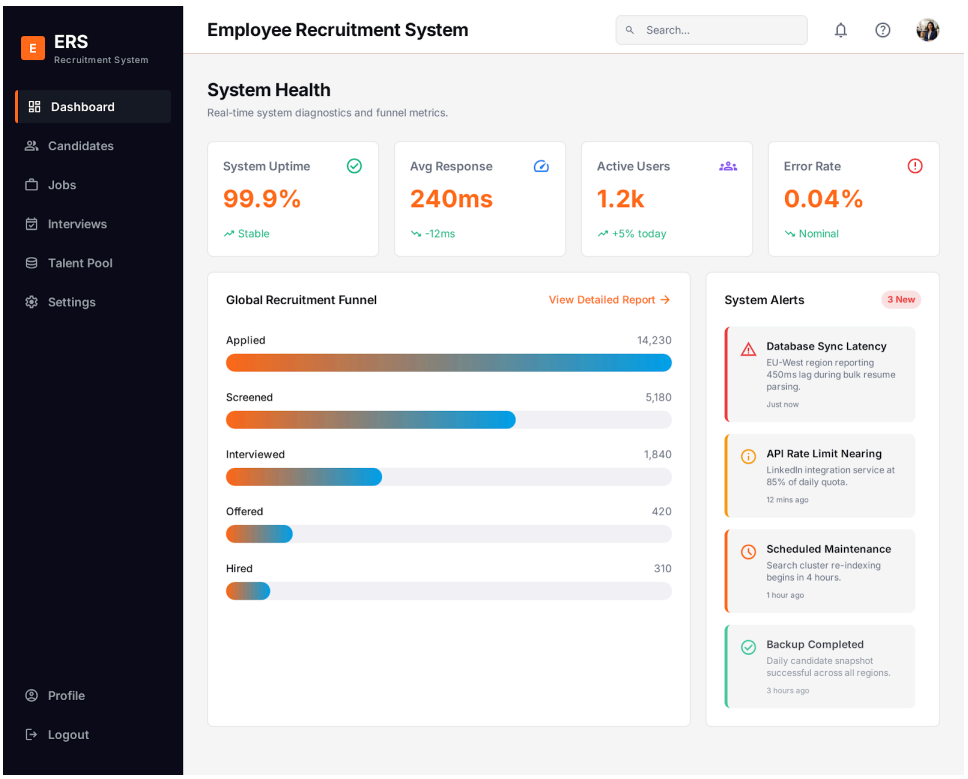



ERS
Recruitment System
Dashboard
Candidates
Jobs
Interviews
Talent Pool
Settings
Profile
Logout
Welcome back, Alex.
Search...
Senior Frontend Developer
ACTIVELY INTERVIEWING
TechCorp Inc.
San Francisco, CA (Remote)
APPLIED
SCREENING
INTERVIEW
OFFER
ONBOARDING
Next Step: Technical Interview
Scheduled for Tomorrow, 10:00 AM PST with Sarah Jenkins (Engineering Manager).
Join Meeting
Profile Views
142
+12%
Last 30 days
Active Apps
3
Currently tracking
Messages
View All
No new messages
You're all caught up! Recruiter messages will appear here.
Compose Message
Upcoming Schedule
OCT
24
Tech Interview - TechCorp
10:00 AM - 11:30 AM
OCT
28
Culture Fit Sync
Pending Confirmation
Recent Documents
Resume_V4_Final.pdf
Uploaded 2 days ago
Tech_Assessment.zip
Submitted yesterday


ERS
Recruitment System
Dashboard
Candidates
Jobs
Interviews
Talent Pool
Settings
Profile
Logout
Search candidates, skills, or jobs...
Create Vacancy
Candidate Pipeline
Manage active candidates across all open roles.
Filter
SUBMITTED
12
Sarah Jenkins
Senior Frontend Dev
1d in stage
SCREENING
5
MC
Marcus Chen
UX Designer
3d in stage
SHORTLISTED
3
No candidates here


ERS
Recruitment System
Dashboard
Candidates
Jobs
Interviews
Talent Pool
Settings
Profile
Logout
Employee Recruitment System
Search...
Today's Queue
OCT 24, 2023 • 4 INTERVIEWS
10:00 AM
IN PROGRESS
Sarah Jenkins
Senior Frontend Engineer
11:30 AM
UPCOMING
Marcus Chen
Product Designer
2:00 PM
UPCOMING
Elena Rodriguez
Backend Developer
INTERVIEW EVALUATION • ROUND 2: TECHNICAL
Sarah Jenkins
Applied for: Senior Frontend Engineer
Save Draft
Submit Evaluation
Core Competencies
Technical Proficiency
React, System Architecture, State Management
8
Needs Work (0)
Exceptional (10)
Problem Solving
Algorithm approach, debugging logic
7
Needs Work (0)
Exceptional (10)
Communication
Clarity, articulation of concepts
9
Needs Work (0)
Exceptional (10)
Teamwork
Collaboration, cultural fit
8
Needs Work (0)
Exceptional (10)
Overall Comments
Enter detailed feedback here. Support scores with specific examples from the interview...
Candidate Snapshot
SJ
Sarah Jenkins
s.jenkins@example.com
EXPERIENCE
6 Years
LOCATION
San Francisco, CA (Remote OK)
RESUME
View Document
Previous Round Context
Round 1: Screening
Pass (7.5/10)
"Strong communication skills, solid baseline technical knowledge. Proceed to technical deep-dive." - R. Adams


ERS
Recruitment System
Dashboard
Candidates
Jobs
Interviews
Talent Pool
Settings
Profile
Logout
Dashboard › Approvals
Search...
Approvals
Review and action candidate progressions.
Pending Approvals
5
+2 since yesterday
Avg Time to Hire
18 days
-3 days
Offer Acceptance %
92%
On Target
Action Required
Michael Chang
Senior Frontend Engineer
FINAL ROUND
Technical Score 94/100
Culture Fit 88/100
Reject
Approve
Sarah Jenkins
Lead Product Manager
FINAL ROUND
Technical Score 85/100
Culture Fit 98/100
Reject
Approve
David Osei
Data Scientist
FINAL ROUND
Technical Score 91/100
Culture Fit 82/100
Reject
Approve



References

1. ISO/IEC/IEEE 29148:2018, Systems and Software Engineering, Life Cycle Processes, Requirements Engineering. ISO/IEC/IEEE, 2018.
2. FastAPI Documentation, "FastAPI," fastapi.tiangolo.com, accessed 2026.
3. PostgreSQL Global Development Group, "PostgreSQL Documentation," postgresql.org, accessed 2026.
4. React Documentation, "React," react.dev, accessed 2026.
5. Tailwind Labs, "Tailwind CSS Documentation," tailwindcss.com, accessed 2026.